Introduction to Python

Chapters 1 – 4 | RAG Training Dataset

<!-- CHUNK_START: chapter="1" section="0" topic="chapter_1_header" difficulty="beginner" type="chapter_header" has_code="false" -->

CHAPTER 1: GETTING STARTED

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.1" topic="what_is_python" difficulty="beginner" type="section_header" has_code="false" -->

1.1 What is Python?

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.1" topic="what_is_python" difficulty="beginner" type="concept" has_code="false" -->

Python is a general-purpose programming language created by Guido van Rossum and first released in 1991. The name comes not from the snake but from the British comedy group Monty Python, which reflects the language's emphasis on fun and readability.

Python is designed to be easy to read and write. Its syntax uses plain English words and meaningful indentation instead of curly braces, so code looks more like a structured essay than a wall of symbols. This makes Python one of the best first languages to learn, but it does not stay a beginner language — professionals use Python for web development, data science, machine learning, automation, scientific computing, and much more.

Python is interpreted, meaning you can run a program line by line without a separate compilation step. This makes experimentation fast: write a line, run it, see what happens. Python is also dynamically typed, so you do not need to declare the type of a variable before using it. The interpreter figures it out at runtime.

Python follows a clear philosophy summarised in a document called the Zen of Python. A few guiding principles: beautiful is better than ugly, explicit is better than implicit, and readability counts. These are not just slogans — they shape how the community writes and reviews code.

The Python ecosystem is enormous. The standard library that ships with Python covers networking, file handling, mathematics, date and time, testing, and more. The third-party ecosystem on PyPI (the Python Package Index) contains hundreds of thousands of additional packages installable with a single command.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.1" topic="what_is_python" difficulty="beginner" type="analogy" has_code="false" -->
Think of Python as a very capable Swiss Army knife. When you first pick it up, the main blade is immediately useful and obvious. As you get more comfortable, you discover the other tools — and eventually you learn that professionals use those same tools to build serious things, not just to tinker.

Compare Python to some other languages: Java requires you to declare types, write classes even for simple tasks, and compile before running. C forces you to manage memory yourself. These are powerful but steep entry points. Python removes those barriers so you can focus on solving the problem rather than fighting the language.

Python code is also highly portable. Write a program on Windows and it will run unchanged on a Mac or Linux machine, because Python handles the platform differences for you.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.2" topic="installing_python" difficulty="beginner" type="section_header" has_code="false" -->
1.2 Installing Python

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.2" topic="installing_python" difficulty="beginner" type="concept" has_code="false" -->
Python is free and available for Windows, macOS, and Linux. The official download is at python.org. Always download the latest stable version (3.x). Python 2 reached end-of-life in 2020 and should not be used for new projects.

On Windows, run the installer and check the box that says "Add Python to PATH" before clicking Install. This checkbox is easy to miss and skipping it means you will have to type the full path to Python every time you use the command line.

On macOS, the system ships with an older Python 2 version. Install Python 3 from python.org or using Homebrew by running: brew install python3

On Linux (Ubuntu/Debian), Python 3 is usually pre-installed. Verify with: python3 --version. If it is missing: sudo apt install python3

After installation, open a terminal (Command Prompt on Windows, Terminal on macOS/Linux) and type: python3 --version. If you see something like Python 3.11.2, installation succeeded.

For writing and running code you need an editor. Two good choices for beginners are IDLE (ships with Python, simple, good for learning) and VS Code (free, widely used by professionals, has excellent Python support via the Python extension from Microsoft). Install VS Code from code.visualstudio.com, then install the Python extension from the Extensions panel.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.2" topic="installing_python" difficulty="beginner" type="code_example" has_code="true" -->
Verifying your installation and running a quick check:

[CODE]

```
# In your terminal or command prompt:
python3 --version
# Expected output: Python 3.x.x

# Open the interactive interpreter:
python3
# You will see the >>> prompt. Type:
>>> 2 + 2
4
>>> exit()
```

[/CODE]

The interactive interpreter (also called the REPL — Read, Evaluate, Print Loop) lets you type Python expressions and see results immediately. It is excellent for experimenting. For anything longer than a few lines you will want to write a .py file instead.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.3" topic="first_program" difficulty="beginner" type="section_header" has_code="false" -->
1.3 Your First Program

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.3" topic="first_program" difficulty="beginner" type="concept" has_code="false" -->
A Python program is a plain text file with a .py extension. You write instructions in it and then ask Python to run them. Every instruction is called a statement. Python reads your file from top to bottom and executes each statement in order.

The simplest useful statement is print(). It displays output to the screen. print() is a built-in function — a named, reusable block of code that Python provides for you. You call it by writing its name followed by parentheses containing what you want to print.

Comments are lines that Python ignores completely. They start with a # character. Use them to leave notes for yourself and for anyone else who reads your code. Good comments explain why something is done, not just what is done.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.3" topic="first_program" difficulty="beginner" type="code_example" has_code="true" -->
Create a file called hello.py and write the following:

[CODE]
```
# This is my first Python program.
# Comments begin with # and are ignored by Python.

print("Hello, World!")
print("My name is Python.")
print("I am", 30, "years old.")
```
[/CODE]

Output:

[CODE]
```
Hello, World!
My name is Python.
I am 30 years old.
```
[/CODE]

To run it, open a terminal, navigate to the folder containing hello.py, and type: python3 hello.py

Notice the last print() call passes three separate values separated by commas. print() joins them with spaces automatically. You can print numbers, text, or a mix of both.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.3" topic="print_function" difficulty="beginner" type="code_example" has_code="true" -->
The print() function has optional parameters that control its behaviour:

[CODE]
```
# Default behaviour: items separated by space, newline at end
print("apple", "banana", "mango")
# Output: apple banana mango

# Change separator with sep=
print("apple", "banana", "mango", sep=", ")
# Output: apple, banana, mango

# Prevent newline at end with end=
print("Loading", end="")
print("...")
# Output: Loading...

# Print an empty line
print()
```
[/CODE]

The sep and end parameters are keyword arguments — you pass them by name. You will learn more about keyword arguments in Chapter 6.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.4" topic="getting_started_mistakes" difficulty="beginner" type="section_header" has_code="false" -->

1.4 Common Mistakes

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.4" topic="getting_started_mistakes" difficulty="beginner" type="common_mistake" has_code="true" -->

Mistake 1: Missing parentheses on print

[CODE]

```
# WRONG (Python 2 syntax, fails in Python 3)
print "Hello"

# CORRECT
print("Hello")
```

[/CODE]

In Python 3, print is a function and always requires parentheses. This trips up people who learned Python 2 or who read older tutorials.

Mistake 2: Mismatched quotes

[CODE]

```
# WRONG — opens with double, closes with single
print("Hello, World!')

# CORRECT — quotes must match
print("Hello, World!")
print('Hello, World!')
```

[/CODE]

Python accepts both single and double quotes but the opening and closing quotes must be the same character.

Mistake 3: Indentation errors before you have learned indentation

[CODE]

```
# WRONG — unexpected indent
print("Hello")
    print("World")   # IndentationError

# CORRECT — top-level statements have no indentation
print("Hello")
print("World")
```

[/CODE]

Python uses indentation to define structure. Stray spaces at the start of a line will cause an IndentationError. This usually happens from accidentally pressing Tab or Space before a statement.

Mistake 4: Forgetting to save the file before running it. If your output looks like an older version of your code, check that you saved the file.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="1" section="1.4" topic="getting_started_mistakes" difficulty="beginner" type="practice_problem" has_code="true" -->

Practice Problem 1.1 — Fix the broken program

The following program has three errors. Find and fix all of them.

[CODE]

```
# Broken version
print "Welcome to Python"
print('The answer is", 42)
    print("Done.")
```

[/CODE]

Expected output:

[CODE]

```
Welcome to Python
The answer is 42
Done.
```

[/CODE]

Hint: Check each of the three common mistakes described above.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="0" topic="chapter_2_header" difficulty="beginner" type="chapter_header" has_code="false" -->

CHAPTER 2: VARIABLES AND DATA TYPES

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.1" topic="variables" difficulty="beginner" type="section_header" has_code="false" -->

2.1 Variables

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.1" topic="variables" difficulty="beginner" type="concept" has_code="false" -->

A variable is a named storage location that holds a value. Think of it as a labelled box: you put something in the box, give the box a name, and later you can refer to the box by that name to get the value back — or replace it with something new.

You create a variable in Python with an assignment statement. The syntax is: name = value. The equals sign here does not mean mathematical equality. It is an instruction: "evaluate the right side and store the result under this name."

Variable names in Python follow these rules. They can contain letters, digits, and underscores. They cannot start with a digit. They cannot contain spaces or special characters like @, -, or !. They are case-sensitive, so score, Score, and SCORE are three different variables. They cannot be Python keywords like if, for, while, return, or True.

By convention, Python programmers use snake_case for variable names: all lowercase with underscores separating words. Examples: player_name, total_score, number_of_items. Avoid short meaningless names like x, a, or tmp unless you are writing a throwaway script or the variable has an obvious mathematical meaning like i for a loop counter.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.1" topic="variables" difficulty="beginner" type="code_example" has_code="true" -->
Creating and using variables:

[CODE]
```
# Assign values to variables
player_name = "Alice"
score = 0
lives = 3
is_playing = True

# Read a variable's value
print(player_name)    # Alice
print(score)          # 0

# Update a variable
score = 100
print(score)          # 100

# Assign the result of an expression
total = score + 50
print(total)          # 150

# Multiple assignment on one line
x = y = z = 0
print(x, y, z)        # 0 0 0

# Swap two variables (Python makes this elegant)
a = 10
b = 20
a, b = b, a
print(a, b)           # 20 10
```
[/CODE]

Variables do not have a fixed type in Python. You can reassign a variable to a completely different kind of value at any time, though doing this carelessly makes code harder to read.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.1" topic="variables" difficulty="beginner" type="analogy" has_code="false" -->
Imagine a whiteboard in a classroom. A variable is like writing a name on the board and drawing a box next to it. You can write anything in the box. Later you can erase the box's contents and write something new, but the name on the board stays the same until you erase that too.

In memory, Python creates an object (the actual value), then makes the variable name point to that object. When you reassign a variable, you are not changing the object — you are moving the pointer to a different object. This distinction matters when you learn about lists and mutability in Chapter 7, but for now, think of variables as simple named containers.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.2" topic="numbers" difficulty="beginner" type="section_header" has_code="false" -->

2.2 Integers and Floats

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.2" topic="numbers" difficulty="beginner" type="concept" has_code="false" -->

Python has two primary numeric types. An integer (int) is a whole number with no decimal point: 0, 1, -5, 1000000. A float is a number with a decimal point: 3.14, -0.5, 2.0. Note that 2 and 2.0 are different types even though they represent the same mathematical value.

Integers in Python can be arbitrarily large. You can compute 2 to the power of 1000 and Python will give you the exact answer. Floats use IEEE 754 double-precision representation, which gives about 15 significant decimal digits of precision. This means floats are fast and memory-efficient but not always exact.

Arithmetic operators: + adds, - subtracts, * multiplies, / divides (always returns a float), // performs integer (floor) division, % gives the remainder (modulo), ** raises to a power.

The type() function tells you what type a value is.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.2" topic="numbers" difficulty="beginner" type="code_example" has_code="true" -->

Working with numbers:

```
[CODE]
a = 10
b = 3

print(a + b)     # 13
print(a - b)     # 7
print(a * b)     # 30
print(a / b)     # 3.3333333333333335  (always a float)
print(a // b)    # 3   (integer division, truncates decimal)
print(a % b)     # 1   (remainder: 10 = 3*3 + 1)
print(a ** b)    # 1000 (10 to the power 3)

# Type checking
print(type(10))       # <class 'int'>
print(type(3.14))     # <class 'float'>
print(type(10 / 2))   # <class 'float'>  -- division always produces float
print(type(10 // 2))  # <class 'int'>    -- floor division keeps int

# Float precision issue (important to know)
print(0.1 + 0.2)      # 0.30000000000000004 -- not exactly 0.3
print(round(0.1 + 0.2, 2))  # 0.3 -- use round() when precision matters
[/CODE]
```

The float precision issue is not a Python bug — it is a fundamental property of how all modern computers store decimal numbers in binary. For financial calculations, use Python's decimal module instead of float.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.3" topic="strings_intro" difficulty="beginner" type="section_header" has_code="false" -->

2.3 Strings

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.3" topic="strings_intro" difficulty="beginner" type="concept" has_code="false" -->

A string is a sequence of characters enclosed in quotes. Python accepts single quotes, double quotes, or triple quotes. Single and double quotes are interchangeable for single-line strings. Triple quotes allow strings that span multiple lines.

Strings are immutable — once created, you cannot change individual characters. Operations on strings always produce new strings rather than modifying the original.

You can join two strings with the + operator (called concatenation) and repeat a string with the * operator. The len() function returns the number of characters in a string. Individual characters are accessed with indexing using square brackets: the first character is at index 0.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.3" topic="strings_intro" difficulty="beginner" type="code_example" has_code="true" -->
Basic string operations:

[CODE]

```
greeting = "Hello"
name = 'World'

# Concatenation
message = greeting + ", " + name + "!"
print(message)          # Hello, World!

# Repetition
print("ha" * 3)         # hahaha

# Length
print(len(message))     # 13

# Indexing (zero-based)
print(message[0])       # H
print(message[7])       # W
print(message[-1])      # !  (negative index counts from end)

# Slicing
print(message[0:5])     # Hello
print(message[7:])      # World!
print(message[:5])      # Hello

# Multi-line string
poem = (
    "Roses are red,
"
    "Violets are blue,
"
    "Python is great,
"
    "And so are you."
)
print(poem)

# Check if substring exists
print("World" in message)   # True
print("Python" in message)  # False
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.4" topic="booleans" difficulty="beginner" type="section_header" has_code="false" -->

2.4 Booleans

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.4" topic="booleans" difficulty="beginner" type="concept" has_code="false" -->

A boolean value is either True or False. These are the only two values of the bool type. Note the capital first letter — true and false (lowercase) are not booleans in Python; they would be interpreted as variable names.

Booleans are the result of comparison operations: == (equal), != (not equal), < (less than), > (greater than), <= (less than or equal), >= (greater than or equal). They are also produced by logical operators: and, or, not.

Every value in Python has a boolean interpretation called its truthiness. Zero (0), empty strings (""), empty lists ([]), and None all evaluate to False. Everything else evaluates to True. You rarely need to write == True or == False explicitly — Python checks the truthiness automatically in conditional statements.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.4" topic="booleans" difficulty="beginner" type="code_example" has_code="true" -->
Boolean values and operations:

[CODE]

```
is_raining = True
is_sunny = False

print(type(True))       # <class 'bool'>
print(type(False))      # <class 'bool'>

# Comparison operators produce booleans
print(5 > 3)            # True
print(5 == 5)           # True
print(5 != 3)           # True
print(10 <= 9)          # False

# Logical operators
print(True and False)   # False
print(True or False)    # True
print(not True)         # False

# Truthiness of other types
print(bool(0))          # False
print(bool(1))          # True
print(bool(""))         # False
print(bool("hello"))    # True
print(bool([]))         # False
print(bool([1, 2]))     # True
print(bool(None))       # False

# Short-circuit evaluation
# 'and' stops at first False, 'or' stops at first True
x = 0
result = x != 0 and (10 / x > 1)  # Safe: second part not evaluated if x==0
print(result)           # False
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.5" topic="type_conversion" difficulty="beginner" type="section_header" has_code="false" -->

2.5 Type Conversion

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.5" topic="type_conversion" difficulty="beginner" type="concept" has_code="false" -->

Type conversion (also called casting) means changing a value from one type to another. Python provides built-in functions for this: int() converts to integer, float() converts to float, str() converts to string, bool() converts to boolean.

There are two kinds of conversion. Implicit conversion happens automatically when Python promotes one type to another during an operation — for example, when you add an int and a float, Python converts the int to a float before adding. Explicit conversion is when you call a conversion function yourself.

Conversion can fail. Calling int("hello") raises a ValueError because "hello" cannot be interpreted as an integer. When converting user input (which always arrives as a string), you should validate the input or use a try/except block (covered in Chapter 11) to handle invalid input gracefully.

The input() function is important here: it always returns a string, even if the user typed a number. You must convert explicitly if you need a number.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.5" topic="type_conversion" difficulty="beginner" type="code_example" has_code="true" -->
Type conversion examples:

[CODE]

```
# String to number
age_string = "25"
age = int(age_string)
print(age + 1)           # 26

price_string = "9.99"
price = float(price_string)
print(price * 2)         # 19.98

# Number to string
score = 100
message = "Your score is " + str(score)
print(message)           # Your score is 100

# Float to int (truncates, does NOT round)
print(int(3.9))          # 3
print(int(-3.9))         # -3

# Getting input from the user
name = input("Enter your name: ")          # always returns str
age = int(input("Enter your age: "))       # convert immediately

# Implicit conversion
result = 3 + 2.0         # int + float -> float
print(result)            # 5.0
print(type(result))      # <class 'float'>
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.6" topic="variables_types_mistakes" difficulty="beginner" type="section_header" has_code="false" -->

2.6 Common Mistakes

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.6" topic="variables_types_mistakes" difficulty="beginner" type="common_mistake" has_code="true" -->

Mistake 1: Concatenating a string and a number without converting

[CODE]
```
# WRONG
age = 25
print("I am " + age + " years old.")   # TypeError

# CORRECT
print("I am " + str(age) + " years old.")
# Or better, use an f-string (Chapter 3):
print(f"I am {age} years old.")
```
[/CODE]

Mistake 2: Comparing with = instead of ==

[CODE]
```
x = 10
if x = 10:      # SyntaxError — = is assignment, not comparison
    print("ten")

if x == 10:     # CORRECT — == tests equality
    print("ten")
```
[/CODE]

Mistake 3: Forgetting that input() always returns a string

[CODE]
```
# WRONG — tries to add string "5" to int 3, raises TypeError
user_input = input("Enter a number: ")
result = user_input + 3

# CORRECT — convert first
result = int(user_input) + 3
```
[/CODE]

Mistake 4: Using a variable before assigning it

[CODE]
```
print(total)    # NameError: name 'total' is not defined
total = 0
```
[/CODE]

Python reads code top to bottom. A variable does not exist until the assignment line executes.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="2" section="2.6" topic="variables_types_mistakes" difficulty="beginner" type="practice_problem" has_code="true" -->

Practice Problem 2.1 — Temperature converter

Write a program that asks the user for a temperature in Celsius and prints it converted to Fahrenheit. The formula is: F = (C * 9/5) + 32

Expected behaviour:

[CODE]

```
Enter temperature in Celsius: 100
100.0 Celsius is 212.0 Fahrenheit

Enter temperature in Celsius: 0
0.0 Celsius is 32.0 Fahrenheit
```

[/CODE]

Practice Problem 2.2 — Type detective

Without running the code, predict the output of each line. Then run it and check.

[CODE]

```
print(type(5))
print(type(5.0))
print(type(5 / 2))
print(type(5 // 2))
print(type("5"))
print(type(True))
print(type(None))
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="0" topic="chapter_3_header" difficulty="beginner" type="chapter_header" has_code="false" -->

CHAPTER 3: OPERATORS AND EXPRESSIONS

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.1" topic="arithmetic_operators" difficulty="beginner" type="section_header" has_code="false" -->

3.1 Arithmetic Operators

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.1" topic="arithmetic_operators" difficulty="beginner" type="concept" has_code="false" -->

An expression is a combination of values, variables, and operators that Python evaluates to produce a result. The simplest expressions are arithmetic: they combine numbers using arithmetic operators and produce a numeric result.

Python's arithmetic operators are: + (addition), - (subtraction), * (multiplication), / (true division — always returns float), // (floor division — rounds down to nearest integer), % (modulo — returns remainder), ** (exponentiation).

Operator precedence determines the order of evaluation when multiple operators appear in one expression. Python follows standard mathematical rules, sometimes remembered as PEMDAS or BODMAS: Parentheses first, then Exponentiation, then Multiplication/Division/Modulo (left to right), then Addition/Subtraction (left to right). When in doubt, use parentheses to make the intended order explicit. Explicit parentheses never hurt readability.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.1" topic="arithmetic_operators" difficulty="beginner" type="code_example" has_code="true" -->

Arithmetic operators and precedence:

[CODE]

```
# Basic arithmetic
print(7 + 3)      # 10
print(7 - 3)      # 4
print(7 * 3)      # 21
print(7 / 3)      # 2.3333333333333335
print(7 // 3)     # 2   (floor division)
print(7 % 3)      # 1   (remainder: 7 = 2*3 + 1)
print(2 ** 8)     # 256 (2 to the power 8)

# Precedence in action
print(2 + 3 * 4)       # 14  (* before +)
print((2 + 3) * 4)     # 20  (parentheses first)
print(2 ** 3 ** 2)     # 512 (** is right-associative: 2**(3**2) = 2**9)
print(10 - 3 - 2)      # 5   (left-to-right: (10-3)-2)

# Practical uses of modulo
print(10 % 2 == 0)     # True — 10 is even
print(15 % 2 == 0)     # False — 15 is odd
print(360 % 360)       # 0   — useful for circular/wrap-around logic

# Augmented assignment (shorthand)
count = 0
count += 1     # same as: count = count + 1
count *= 2     # same as: count = count * 2
print(count)   # 2
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.2" topic="comparison_operators" difficulty="beginner" type="section_header" has_code="false" -->

3.2 Comparison Operators

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.2" topic="comparison_operators" difficulty="beginner" type="concept" has_code="false" -->

Comparison operators compare two values and always return a boolean (True or False). They are: == (equal to), != (not equal to), < (less than), > (greater than), <= (less than or equal to), >= (greater than or equal to).

A common error is confusing = (assignment) with == (comparison). The single equals sign stores a value; the double equals sign tests whether two values are equal.

Python allows chained comparisons, which is a feature not found in most other languages. You can write 0 < x < 10 instead of x > 0 and x < 10. Python evaluates chained comparisons left to right and short-circuits — if one part is False, the rest are not evaluated.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.2" topic="comparison_operators" difficulty="beginner" type="code_example" has_code="true" -->

Comparison operators:

[CODE]

```
x = 7

print(x == 7)     # True
print(x == 8)     # False
print(x != 8)     # True
print(x > 5)      # True
print(x < 5)      # False
print(x >= 7)     # True
print(x <= 6)     # False

# Chained comparisons
age = 25
print(18 <= age < 65)    # True — adult working age

score = 75
grade_b = 70 <= score < 80
print(grade_b)           # True

# Comparing strings (lexicographic / alphabetical order)
print("apple" < "banana")    # True
print("Zebra" < "apple")     # True — uppercase letters come before lowercase in ASCII
print("apple" == "apple")    # True
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.3" topic="logical_operators" difficulty="beginner" type="section_header" has_code="false" -->

3.3 Logical Operators

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.3" topic="logical_operators" difficulty="beginner" type="concept" has_code="false" -->

Logical operators combine boolean expressions. Python has three: and, or, not.

The and operator returns True only if both operands are True. If the left operand is False, Python does not evaluate the right operand at all — this is called short-circuit evaluation. It is a useful property: you can guard against errors by putting the safety check on the left side.

The or operator returns True if at least one operand is True. If the left operand is True, Python does not evaluate the right operand.

The not operator reverses a boolean: not True is False, not False is True.

Operator precedence for boolean expressions: not is evaluated first, then and, then or. Use parentheses when combining multiple logical operators to make your intent clear.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.3" topic="logical_operators" difficulty="beginner" type="code_example" has_code="true" -->
Logical operators:

[CODE]

```
# and — both must be True
print(True and True)     # True
print(True and False)    # False
print(False and True)    # False

# or — at least one must be True
print(True or False)     # True
print(False or False)    # False

# not — reverses the boolean
print(not True)          # False
print(not False)         # True

# Practical example
age = 20
has_id = True

can_enter = age >= 18 and has_id
print(can_enter)         # True

# Short-circuit safety
items = []
first = len(items) > 0 and items[0]   # Safe: items[0] not accessed if list is empty
print(first)             # False

# not with comparisons
x = 5
print(not x == 5)        # False
print(x != 5)            # False -- same result, more readable
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.4" topic="assignment_operators" difficulty="beginner" type="section_header" has_code="false" -->
3.4 Assignment Operators

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.4" topic="assignment_operators" difficulty="beginner" type="concept" has_code="false" -->
Python's basic assignment operator is =. In addition, Python provides augmented assignment operators that combine an arithmetic operation with assignment. These are shorthand forms that make code more concise. All of the following have an augmented form: += (add and assign), -= (subtract and assign), *= (multiply and assign), /= (divide and assign), //= (floor-divide and assign), %= (modulo and assign), **= (exponentiate and assign).

Walrus operator: Python 3.8 introduced :=, called the walrus operator. It assigns a value as part of an expression, allowing you to assign and test in one step. It is most useful inside while loops and comprehensions.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.4" topic="assignment_operators" difficulty="beginner" type="code_example" has_code="true" -->
Assignment operators:

[CODE]

```
# Standard augmented assignment
score = 100
score += 50     # score = score + 50
print(score)    # 150

score -= 20
print(score)    # 130

score *= 2
print(score)    # 260

score //= 3
print(score)    # 86

# Walrus operator := (Python 3.8+)
# Traditional: assign, then test
line = input()
while line != "quit":
    print("You said:", line)
    line = input()

# With walrus: assign inside the condition
while (line := input()) != "quit":
    print("You said:", line)
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.5" topic="fstrings" difficulty="beginner" type="section_header" has_code="false" -->
3.5 F-strings

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.5" topic="fstrings" difficulty="beginner" type="concept" has_code="false" -->
An f-string (formatted string literal) is the modern, readable way to embed variable values and expressions directly inside a string. You create one by putting an f before the opening quote, then placing expressions inside curly braces {}.

F-strings were introduced in Python 3.6 and are now the preferred way to format strings. Earlier alternatives include the % operator and str.format(), but f-strings are shorter, faster, and easier to read.

Inside the curly braces you can put any valid Python expression — a variable, a calculation, a function call, even a conditional. You can also add format specifiers after a colon to control number formatting, padding, and alignment.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.5" topic="fstrings" difficulty="beginner" type="code_example" has_code="true" -->
F-string usage:

[CODE]

```
name = "Alice"
score = 95
pi = 3.14159265

# Basic embedding
print(f"Hello, {name}!")                # Hello, Alice!
print(f"{name} scored {score} points.") # Alice scored 95 points.

# Expressions inside f-strings
print(f"Double score: {score * 2}")     # Double score: 190
print(f"Pass? {score >= 60}")           # Pass? True

# Format specifiers
print(f"Pi is approximately {pi:.2f}")  # Pi is approximately 3.14
print(f"Score: {score:>10}")            # Score:         95  (right-align in 10 chars)
print(f"Score: {score:<10}!")           # Score: 95        !  (left-align)
print(f"Score: {score:0>5}")            # Score: 00095  (pad with zeros)

# Older alternatives (you may see these in existing code)
print("Hello, %s! Score: %d" % (name, score))    # % formatting
print("Hello, {}! Score: {}".format(name, score)) # str.format()
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.6" topic="operators_mistakes" difficulty="beginner" type="section_header" has_code="false" -->

3.6 Common Mistakes

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.6" topic="operators_mistakes" difficulty="beginner" type="common_mistake" has_code="true" -->

Mistake 1: Integer division surprise

[CODE]

```
# In Python 3, / always returns a float
print(10 / 2)      # 5.0, not 5

# Use // if you want an integer result
print(10 // 2)     # 5
```

[/CODE]

Mistake 2: Operator precedence surprise

[CODE]

```
# WRONG assumption: thinking left-to-right
print(2 + 3 * 4)   # 14, not 20
# * has higher precedence than +

# When unsure, use parentheses
print((2 + 3) * 4)  # 20 — intention is now explicit
```

[/CODE]

Mistake 3: Chained comparison misread

[CODE]

```
# This looks like it tests if x equals both 1 and 2 simultaneously
# It actually chains: x==1 and 1==2
x = 1
print(x == 1 == 2)   # False (1==2 is False)
print(x == 1 and x == 2)  # False (clearer intent)
```

[/CODE]

Mistake 4: Forgetting f before f-string

[CODE]

```
name = "Alice"
print("Hello, {name}")    # Hello, {name}  -- literal text, not substituted
print(f"Hello, {name}")   # Hello, Alice   -- f-string evaluates {}
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="3" section="3.6" topic="operators_mistakes" difficulty="beginner" type="practice_problem" has_code="true" -->
Practice Problem 3.1 — Grade calculator

Write a program that: 1. Asks the user for three test scores 2. Calculates the average 3. Prints the average formatted to one decimal place 4. Prints the letter grade: A (90+), B (80-89), C (70-79), D (60-69), F (below 60)

Example output:

[CODE]

```
Enter score 1: 85
Enter score 2: 92
Enter score 3: 78
Average: 85.0
Grade: B
```

[/CODE]

Practice Problem 3.2 — Even/odd and divisibility

Write a program that takes an integer and prints: - Whether it is even or odd - Whether it is divisible by 3 - Whether it is divisible by both 2 and 3

Use f-strings for all output.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="0" topic="chapter_4_header" difficulty="beginner" type="chapter_header" has_code="false" -->
CHAPTER 4: CONTROL FLOW

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.1" topic="if_statement" difficulty="beginner" type="section_header" has_code="false" -->
4.1 The if Statement

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.1" topic="if_statement" difficulty="beginner" type="concept" has_code="false" -->
Control flow refers to the order in which Python executes statements. By default Python runs each line once, top to bottom. Control flow structures let you change that: skipping some lines, repeating others, or choosing between alternatives.

The if statement executes a block of code only when a condition is True. The condition is any expression that evaluates to a boolean. The indented block that follows is called the body of the if. Python uses indentation (consistently 4 spaces per level, not tabs) to define where the body begins and ends. This is not just a style convention — it is part of the language syntax.

After the body, execution continues with the first unindented line. If the condition was False, the body was skipped and that first unindented line is the next thing to run.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.1" topic="if_statement" difficulty="beginner" type="code_example" has_code="true" -->
The if statement:

[CODE]
```
age = 20

if age >= 18:
    print("You are an adult.")
    print("You may vote.")

print("This line always runs.")

# Output:
# You are an adult.
# You may vote.
# This line always runs.

# If condition is False, body is skipped entirely
temperature = 15

if temperature > 30:
    print("It's hot today.")
    print("Drink water.")

print("Have a nice day.")

# Output:
# Have a nice day.
```
[/CODE]

The indentation level of the body lines is critical. All lines in the body must be indented by the same amount. Python raises an IndentationError if the indentation is inconsistent.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.2" topic="if_else" difficulty="beginner" type="section_header" has_code="false" -->
4.2 if-else

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.2" topic="if_else" difficulty="beginner" type="concept" has_code="false" -->
An if-else statement provides two branches: one that runs when the condition is True, another that runs when the condition is False. Exactly one of the two branches always executes — never both, never neither.

The else clause has no condition of its own. It is the default: whatever did not match the if condition falls through to else. The else block is indented at the same level as the if keyword, not inside the if body.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.2" topic="if_else" difficulty="beginner" type="code_example" has_code="true" -->
if-else:

[CODE]

```
score = 72

if score >= 60:
    print("Pass")
else:
    print("Fail")
# Output: Pass

# Another example
number = 7

if number % 2 == 0:
    print(f"{number} is even")
else:
    print(f"{number} is odd")
# Output: 7 is odd

# Ternary expression — one-line if-else for simple cases
status = "adult" if age >= 18 else "minor"
print(status)
# This is concise but use it only for simple conditions
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.3" topic="if_elif_else" difficulty="beginner" type="section_header" has_code="false" -->
4.3 if-elif-else

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.3" topic="if_elif_else" difficulty="beginner" type="concept" has_code="false" -->
When you have more than two possible outcomes, use elif (short for "else if"). You can chain as many elif clauses as needed. Python checks conditions from top to bottom and runs the body of the first condition that is True. Once a match is found, all remaining elif and else clauses are skipped.

The final else is optional. If you include it, it catches everything that did not match any condition above. If you omit it and nothing matches, Python simply does nothing.

Structure: the if, all elifs, and the optional else must all be at the same indentation level. Their bodies are indented one level deeper.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.3" topic="if_elif_else" difficulty="beginner" type="code_example" has_code="true" -->
if-elif-else:

[CODE]

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(f"Score: {score}, Grade: {grade}")
# Output: Score: 85, Grade: B

# Important: order matters
# Python stops at the first True condition
x = 15

if x > 10:
    print("greater than 10")   # This runs
elif x > 5:
    print("greater than 5")    # This is skipped (already matched above)
else:
    print("5 or less")         # This is skipped

# BMI category example
bmi = 22.5

if bmi < 18.5:
    category = "Underweight"
elif bmi < 25.0:
    category = "Normal weight"
elif bmi < 30.0:
    category = "Overweight"
else:
    category = "Obese"

print(f"BMI {bmi}: {category}")
# Output: BMI 22.5: Normal weight
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.4" topic="nested_conditions" difficulty="beginner" type="section_header" has_code="false" -->

4.4 Nested Conditions

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.4" topic="nested_conditions" difficulty="beginner" type="concept" has_code="false" -->

You can place an if statement inside another if statement's body. This is called nesting. Each level of nesting adds another level of indentation. Nested conditions are used when a decision depends on multiple layers of criteria.

Be careful with deep nesting. More than two or three levels becomes hard to read and debug. When you find yourself nesting deeply, consider whether you can restructure using elif, logical operators, or separate functions to flatten the logic.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.4" topic="nested_conditions" difficulty="beginner" type="code_example" has_code="true" -->

Nested conditions:

[CODE]

```
age = 25
has_license = True
has_insurance = False

if age >= 16:
    if has_license:
        if has_insurance:
            print("You may drive.")
        else:
            print("You need insurance first.")
    else:
        print("You need a license first.")
else:
    print("You are too young to drive.")
# Output: You need insurance first.

# Flattened version using 'and' — often cleaner
if age >= 16 and has_license and has_insurance:
    print("You may drive.")
elif age < 16:
    print("You are too young to drive.")
elif not has_license:
    print("You need a license first.")
else:
    print("You need insurance first.")
```

[/CODE]

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.4" topic="nested_conditions" difficulty="beginner" type="analogy" has_code="false" -->
Think of a decision tree at an airport check-in. First question: Do you have a valid passport? If no, you stop there. If yes, second question: Is your flight within the next two hours? Depending on that, you go to the fast-track lane or the regular lane. And so on. Each question only makes sense after the previous one was answered — that is nested conditional logic.

The flattened version using logical operators is like asking: "Can you go through fast-track?" as a single compound question with the full criteria stated at once. Both approaches work; use whichever makes the logic clearer for your specific situation.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.5" topic="control_flow_mistakes" difficulty="beginner" type="section_header" has_code="false" -->

4.5 Common Mistakes

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.5" topic="control_flow_mistakes" difficulty="beginner" type="common_mistake" has_code="true" -->

Mistake 1: Using = instead of == in a condition

[CODE]

```
x = 5
if x = 10:       # SyntaxError: cannot assign inside if condition
    print("ten")

if x == 10:      # CORRECT
    print("ten")
```

[/CODE]

Mistake 2: Forgetting the colon at the end of if/elif/else

[CODE]

```
if x > 0        # SyntaxError: missing colon
    print("positive")

if x > 0:       # CORRECT
    print("positive")
```

[/CODE]

Mistake 3: Incorrect indentation

[CODE]

```
if x > 0:
print("positive")    # IndentationError: expected an indented block

if x > 0:
    print("positive")  # CORRECT: 4 spaces
```

[/CODE]

Mistake 4: Assuming elif is checked after a match

[CODE]

```
x = 15
if x > 10:
    print("big")
elif x > 5:
    print("medium")   # Never prints even if x > 5, because x > 10 matched first
```

[/CODE]

Once a condition matches in an if/elif chain, Python exits the whole chain. Subsequent elif clauses are not checked. Order your conditions from most specific to most general.

<!-- CHUNK_END -->

<!-- CHUNK_START: chapter="4" section="4.5" topic="control_flow_mistakes" difficulty="beginner" type="practice_problem" has_code="true" -->
Practice Problem 4.1 — Login validator

Write a program that simulates a simple login. Store a correct username and password. Ask the user to enter both. Print one of these messages: - "Login successful" if both are correct - "Wrong password" if the username is correct but password is wrong - "Unknown user" if the username is not recognised

Practice Problem 4.2 — Tax bracket calculator

In a fictional country, income tax is calculated as follows: - Income up to 10,000: 0% tax - Income 10,001 to 40,000: 10% on the amount over 10,000 - Income 40,001 to 80,000: 20% on the amount over 40,000 (plus tax from lower brackets) - Income above 80,000: 30% on the amount over 80,000 (plus tax from lower brackets)

Write a program that asks for income and prints the total tax owed and effective tax rate.

Example:

```
[CODE]
Enter your income: 50000
Tax bracket breakdown:
  0% on first 10000: 0
  10% on 30000: 3000
  20% on 10000: 2000
Total tax: 5000
Effective rate: 10.0%
[/CODE]
```

<!-- CHUNK_END -->